

School of Informatics & Cybersecurity - Assessment Submission Coversheet

Student Information

Student Number:	B00173757
Student Name:	Sean Wogan

Module Information

Module Title:	Security Administration
Module Code:	COMP H2703

Assessment Information

Assessment Title:	Project
-------------------	---------

Declaration

☐ I confirm that I have read and understood the University's Academic Integrity Policy.

<https://www.tudublin.ie/explore/about-the-university/academic-affairs/academic-quality-assurance-and-enhancement/academic-integrity/>

☐ I have completed the TU Dublin AI Driving License theory test.

<https://www.tudublin.ie/explore/about-the-university/academic-affairs/ita/ita-resources/artificial-intelligence-driving-license/>

☐ I declare that this assessment submission is my own work.

☐ I confirm that I have read and understood the AI traffic light system used as part of this assessment brief and I have adhered to the level of AI usage allowed for this assessment.

Student signature:	Sean Wogan
Date:	09/12/2025 Contents 1 – SummaryPg. 1 2 – Architecture Overview.....Pg. 2 3 – Security Implementation.....Pg. 5 4 – Bash ScriptingPg. 10 5 – Access ControlPg. 10 6 – Additional Security MeasuresPg.12 6 – ChallengesPg.12 7 – ConclusionPg. 13

1. SUMMARY

My 'Techstart Solutions' project revolved around designing, deploying and applying suitable security measures to a cloud based web application being hosted on Azure Portal. Demonstrating security controls that were added on the network, OS and application layers was the aim for this project. A virtual machine using Ubuntu Linux running on Azure, an SQL database and a web application using Flask were the main structures in my system. Everything was based within an Virtual Network on Azzure with segregated subnets all controlled with NSGs.

I used a range of methods to achieve the security measures I was aiming for:

- **NSGs & UFW**
 - *Network Security Groups on Azure were put in place to apply rules for inbound & outbound traffic for the virtual network and subnet level.*
 - *On the virtual machine, UFW was used to add a default deny policy along with only allowing ports needed such as SSH (22), HTTP(80) & HTTPS(443/5000).*
- **Fail2Ban**
 - *I installed Fail2Ban and used it to protect the system against brute force attacks by configuring it to block IP addresses that exceed the limit of failed logins and monitor SSH login attempts.*
- **HTTPS & Transport Layer Security**
 - *I created a self sign certificate and enabled TLS in Flask to encrypt all web traffic using HTTPS, basically protecting login details and data in transit*
- **Transparent Data Encryption**
 - *SQL Transparent Data Encryption on Azure was enabled to encrypt database data at rest*
- **Unattended-Upgrades**
 - *Automatic security updates were enabled to make sure important patches are installed automatically*
- **Linux Logging**
 - *For system logging I used system-journald and rsyslog for keeping track of events related to security and authentication attempts.*
- **BASH Script**
 - *I wrote a bash script for secure backups of web server files, this gives protection from losing data and system recovery*

- **SQL Backups**

- *Aswell as the Bash backup, I enabled SQL automated backups on Azure to make sure database records are backed up often with the possibility of being restored if there was anything to harm them such as human error, corruption or attacks.*

- **Password Rules**

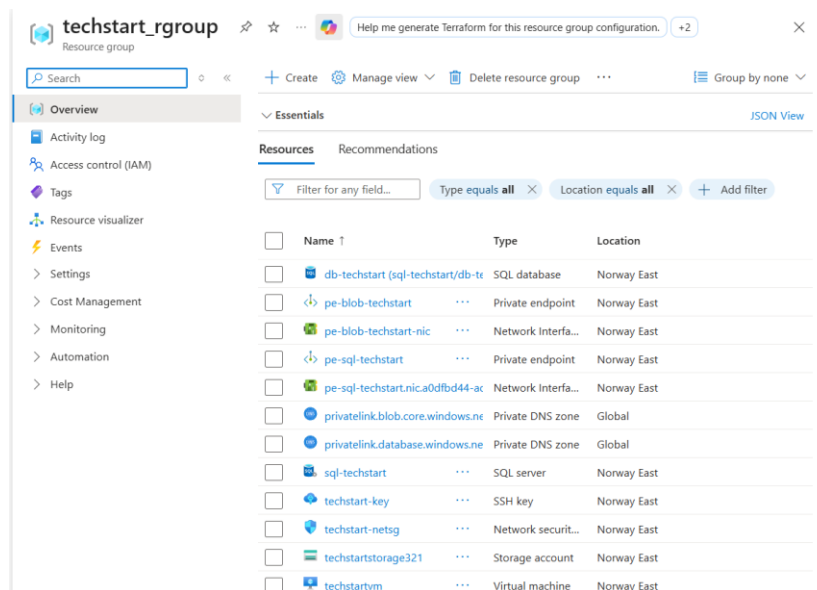
- *Policies for strong password were put in place at both the OS and database levels such as minimum password length requirements and using weak passwords forbidden.*

- **SSH Key Authentication**

- *Instead of password based SSH access I configured key based authentication for the VM which would reduce the probability of unauthorized users accessing it*

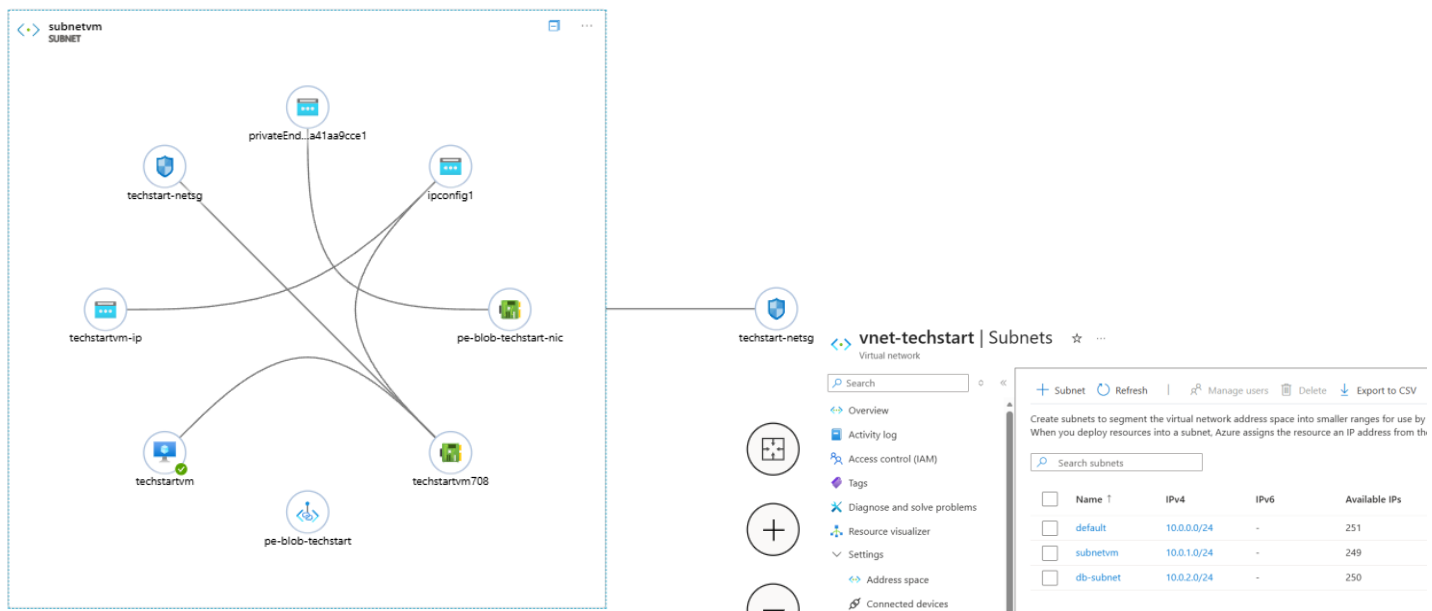
2. ARCHITECTURE OVERVIEW

To begin, I built and deployed my infrastructure using Azure, this was done around a secure, segmented virtual network architecture. All of my resources were contained in a resource group ensuring all of my security configurations can be controlled from one environment.



Name	Type	Location
db-techstart (sql-techstart/db-td)	SQL database	Norway East
pe-blob-techstart	Private endpoint	Norway East
pe-blob-techstart-nic	Network Interface	Norway East
pe-sql-techstart	Private endpoint	Norway East
pe-sql-techstart.nica0dfbd44-ax	Network Interface	Norway East
privatelink.blob.core.windows.net	Private DNS zone	Global
privatelink.database.windows.net	Private DNS zone	Global
sql-techstart	SQL server	Norway East
techstart-key	SSH key	Norway East
techstart-netsg	Network security group	Norway East
techstartstorage321	Storage account	Norway East
techstartvm	Virtual machine	Norway East

Next up I put a Virtual Network in place to allow the virtual machine and database to communicate with each other and provides isolation for the internal resources. Subnets are used inside to separate components such as the virtual machine and database.



Following this, we enforced traffic filtering, allowing ports 22, 443(primary access) & 5000(for flask development only) using a Network Security Group, all other inbound traffic is denied by default.

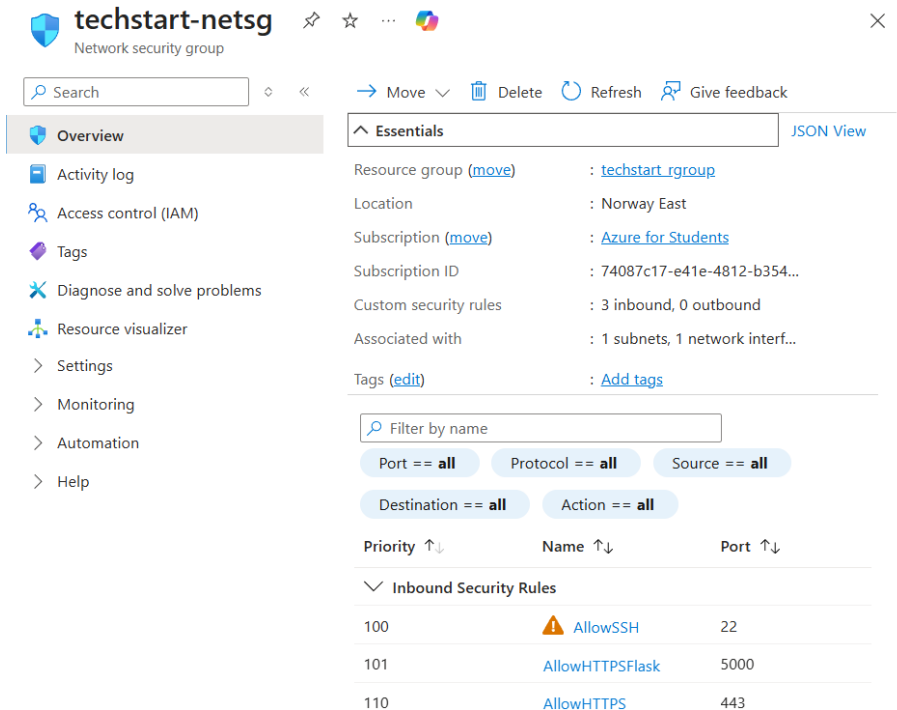


Figure 1 - Network Security Group inbound rules applied to the virtual machine

The Ubuntu Linux Virtual Machine serves as the web application server and is also the point of main security enforcement. It was used to introduce a UFW firewall, Fail2Ban for SSH brute force protection, unattended upgrades and 'systemd-journald' & 'rsyslog'. Here SSH access is restricted with key-based authentication.

```
azureuser@techstartvm:~$ sudo ufw status
Status: active

To Action From
--
OpenSSH ALLOW Anywhere
443 ALLOW Anywhere
80 ALLOW Anywhere
OpenSSH (v6) ALLOW Anywhere (v6)
443 (v6) ALLOW Anywhere (v6)
80 (v6) ALLOW Anywhere (v6)
```

Figure 2 - UFW firewall status showing default deny policy with explicit allow rules

SQL database stored all user data, TLS and TDE were enabled to protect the stored data at rest. A private endpoint was applied to only give access to the SQL database from the VM

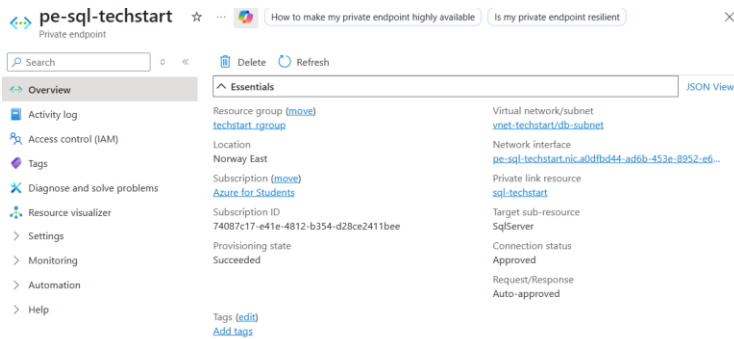


Figure 3 - Azure SQL firewall and private endpoint configuration restricting public access.

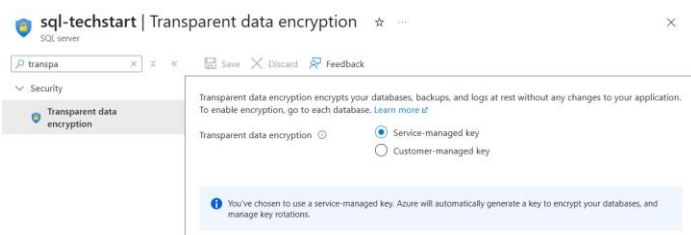


Figure 4 - SQL Database configuration showing Transparent Data Encryption enabled.

All of these create a layered architecture ensuring encrypted communication, defense-in-depth and isolation

3. SECURITY IMPLEMENTATION

To further expand on the security systems we used to protect our web application, I put controls in place in many layers including network, OS and database. Here we will dive into these security controls.

3.1 NSGs & UFW

I configured NSGs at the subnet and interface levels, restricting inbound traffic to only essential ports such as 22 for secure administration, 443 for encrypted web traffic and 5000 for application access. All other ports were denied.

Similar to the NSG rules, I enabled UFW with a default deny policy.

				To	Action	From
				--	-----	----
				22/tcp (OpenSSH)	ALLOW IN	Anywhere
				443	ALLOW IN	Anywhere
				80	ALLOW IN	Anywhere
				5000/tcp	ALLOW IN	Anywhere
100	 AllowSSH	22	TCP	22/tcp (OpenSSH (v6))	ALLOW IN	Anywhere (v6)
101	AllowHTTPSFalk	5000	TCP	443 (v6)	ALLOW IN	Anywhere (v6)
110	AllowHTTPS	443	TCP	80 (v6)	ALLOW IN	Anywhere (v6)
				5000/tcp (v6)	ALLOW IN	Anywhere (v6)

This layered firewall approach ensures that both cloud level and host based traffic controls are enforced, significantly reducing the system's exposed attack surface.

3.2 SSH Hardening & Key-Based Authentication

To aid in preventing unauthorized access I disabled and instead enabled key-based authentication for VM access. This reduces the risk of brute force attacks or password breaches.

```

PermitRootLogin no
#StrictModes yes
#MaxAuthTries 6
#MaxSessions 10

PubkeyAuthentication yes

# Expect .ssh/authorized_keys2 to be
#AuthorizedKeysFile .ssh/aut

#AuthorizedPrincipalsFile none

#AuthorizedKeysCommand none
#AuthorizedKeysCommandUser nobody

# For this to work you will also
#HostbasedAuthentication no
# Change to yes if you don't trust
# HostbasedAuthentication
#IgnoreUserKnownHosts no
# Don't read the user's ~/.rhosts
#IgnoreRhosts yes

# To disable tunneled clear text
PasswordAuthentication no
  
```

Figure 5 - SSH configuration file showing key-based authentication enabled and password login disabled

```

* Support:      https://ubuntu.com/pro

System information as of Fri Dec 12 19:30:52 UTC 2025

System load:  0.0      Processes:      114
Usage of /:   8.0% of 28.02GB  Users logged in:  0
Memory usage: 37%      IPv4 address for eth0: 10.0.1.4
Swap usage:   0%

* Strictly confined Kubernetes makes edge and IoT secure. Learn how MicroK8s
  just raised the bar for easy, resilient and secure K8s cluster deployment

  https://ubuntu.com/engage/secure-kubernetes-at-the-edge

Expanded Security Maintenance for Applications is not enabled.

5 updates can be applied immediately.
To see these additional updates run: apt list --upgradable

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

Last login: Tue Dec  9 11:36:13 2025 from 80.233.42.231
azureuser@techstartvm:~$ |
PS C:\Users\seanw\Downloads\azurekeys> ssh -i techstart-key.pem azureuser@51.13.184.233

```

Figure 6 - Successful SSH key-based login to the virtual machine.

Key-based authentication eliminates reliance on passwords and provides stronger protection against credential theft and automated login attacks.

3.3 Fail2Ban

To add to SSH protection, I installed and configured authentication logs. IP addresses exceeding the allowed number of failed login attempts were automatically banned.

```

azureuser@techstartvm:~$ sudo fail2ban-client status
Status
|- Number of jail:      1
'-- Jail list:        sshd
azureuser@techstartvm:~$ sudo fail2ban-client status sshd
Status for the jail: sshd
|- Filter
| |- Currently failed: 0
| |- Total failed:     0
| '-- Journal matches: _SYSTEMD_UNIT=sshd.service + _COMM=sshd
'-- Actions
| |- Currently banned: 1
| |- Total banned:     1
| '-- Banned IP list:  165.232.81.238

```

Figure 7 - Successful execution of the backup script showing creation of a compressed archive.

By responding automatically to suspicious login behavior, Fail2Ban reduces administrative overhead while improving system availability and security

3.4 HTTPS & TLS

A self signed TLS certificate was generated and configured within the Flask web application. HTTPS was enforced to encrypt all web traffic, protecting login credentials and session data in transit.

```
if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000, ssl_context=("cert.pem", "key.pem"))
```

Figure 8 -Flask web application configured to use HTTPS with a self-signed TLS certificate.

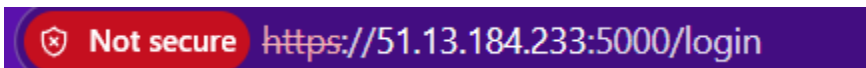


Figure 9 - Browser warning indicating use of a self-signed certificate, confirming encrypted communication.

The not secure label proves:

- HTTPS is enabled
- TLS is active
- The certificate is self-signed & untrusted by browser

Encrypting web traffic ensures that sensitive information such as login credentials cannot be intercepted during transmission across untrusted networks

3.5 System Logging & Monitoring

System logging was implemented using systemd-journald and rsyslog to capture authentication events, service activity, and security-relevant logs.

```
(venv) azureuser@techstartvm:~/techstart-app$ sudo journalctl -n 30
Dec 12 19:42:36 techstartvm systemd[40588]: Reached target default.target - Main
Dec 12 19:42:36 techstartvm systemd[40588]: Startup finished in 219ms.
Dec 12 19:42:36 techstartvm systemd[1]: Started user@1000.service - User Manag
Dec 12 19:42:36 techstartvm systemd[1]: Started session-158.scope - Session 158
Dec 12 19:42:42 techstartvm sshd[40666]: Received disconnect from 193.46.255.7
Dec 12 19:42:42 techstartvm sshd[40666]: Disconnected from authenticating user
Dec 12 19:43:36 techstartvm sshd[40655]: Received disconnect from 109.78.21.182
Dec 12 19:43:36 techstartvm sshd[40655]: Disconnected from user azureuser 109.7
Dec 12 19:43:36 techstartvm sshd[40584]: pam_unix(sshd:session): session closed
Dec 12 19:43:36 techstartvm systemd-logind[1027]: Session 158 logged out. Waiti
Dec 12 19:43:36 techstartvm systemd-logind[1027]: Removed session 158.
Dec 12 19:43:38 techstartvm sshd[40669]: Accepted publickey for azureuser from
Dec 12 19:43:38 techstartvm sshd[40669]: pam_unix(sshd:session): session opened
Dec 12 19:43:38 techstartvm systemd-logind[1027]: New session 160 of user azure
Dec 12 19:43:38 techstartvm systemd[1]: Started session-160.scope - Session 160
Dec 12 19:45:01 techstartvm cron[40739]: Permission denied
Dec 12 19:45:01 techstartvm CRON[40739]: Permission denied
Dec 12 19:48:10 techstartvm systemd[40588]: launchpadlib-cache-clean.service -
Dec 12 19:50:10 techstartvm systemd[1]: Starting sysstat-collect.service - sys
Dec 12 19:50:10 techstartvm systemd[1]: sysstat-collect.service: Deactivated su
Dec 12 19:50:10 techstartvm systemd[1]: Finished sysstat-collect.service - sys
Dec 12 19:50:14 techstartvm sshd[40750]: Connection closed by authenticating us
Dec 12 19:55:01 techstartvm cron[40758]: Permission denied
lines 1-24...skipping...
(venv) azureuser@techstartvm:~/techstart-app$ sudo journalctl _COMM=sshd -n 20
Dec 12 19:34:35 techstartvm sshd[40538]: Disconnected from invalid user 62.60.
Dec 12 19:34:42 techstartvm sshd[40540]: Connection closed by 167.172.77.113 po
Dec 12 19:34:43 techstartvm sshd[40541]: Connection closed by authenticating us
Dec 12 19:36:42 techstartvm sshd[40550]: Invalid user admin from 139.19.117.131
Dec 12 19:36:42 techstartvm sshd[40550]: userauth_pubkey: signature algorithm s
Dec 12 19:36:51 techstartvm sshd[40550]: Connection closed by invalid user admi
Dec 12 19:42:33 techstartvm sshd[40582]: Received disconnect from 193.46.255.10
Dec 12 19:42:33 techstartvm sshd[40582]: Disconnected from authenticating user
Dec 12 19:42:36 techstartvm sshd[40584]: Accepted publickey for azureuser from
Dec 12 19:42:36 techstartvm sshd[40584]: pam_unix(sshd:session): session opened
Dec 12 19:42:42 techstartvm sshd[40666]: Received disconnect from 193.46.255.7
Dec 12 19:43:36 techstartvm sshd[40655]: Received disconnect from 109.78.21.182
Dec 12 19:43:36 techstartvm sshd[40655]: Disconnected from user azureuser 109.7
Dec 12 19:43:36 techstartvm sshd[40584]: pam_unix(sshd:session): session closed
Dec 12 19:43:38 techstartvm sshd[40669]: Accepted publickey for azureuser from
Dec 12 19:43:38 techstartvm sshd[40669]: pam_unix(sshd:session): session opened
Dec 12 19:50:14 techstartvm sshd[40750]: Connection closed by authenticating us
Dec 12 19:55:47 techstartvm sshd[40760]: Received disconnect from 193.46.255.7
Dec 12 19:55:47 techstartvm sshd[40760]: Disconnected from authenticating user
```

Figure 10 - systemd-journald output showing authentication and system log entries.

Centralized logging supports incident investigation and enables administrators to identify abnormal or malicious activity in a timely manner.

3.6 Bash Script

I created a Bash script to do secure backups of web server files. The script archives directories and stores them securely for recovery purposes.

```
GNU nano 7.2 /home/azureuser/secure_backup.sh
#!/bin/bash

# TechStart Secure Backup Script
# Backs up the web application and logs the action

BACKUP_DIR="/var/backups/techstart"
SOURCE_DIR="/home/azureuser/techstart-app"
LOG_FILE="/var/log/techstart_backup.log"
TIMESTAMP=$(date +%Y-%m-%d_%H-%M-%S)

# Create backup directory if it doesn't exist
mkdir -p $BACKUP_DIR

# Create the backup
tar -czf $BACKUP_DIR/techstart_backup_$TIMESTAMP.tar.gz $SOURCE_DIR

# Log the backup
echo "Backup created at $TIMESTAMP" >> $LOG_FILE
```

Figure 11 - Bash script used to automate secure backups of web server files

```
azureuser@techstartvm:~$ ls -lh /var/backups
total 1.3M
-rw-r--r-- 1 root root 50K Dec 9 00:00 alternatives.tar.0
-rw-r--r-- 1 root root 43K Dec 8 23:00 apt.extended_states.0
-rw-r--r-- 1 root root 4.6K Dec 8 20:43 apt.extended_states.1.gz
-rw-r--r-- 1 root root 4.0K Oct 21 04:27 apt.extended_states.2.gz
-rw-r--r-- 1 root root 0 Dec 12 00:00 dpkg.arch.0
-rw-r--r-- 1 root root 32 Dec 11 00:00 dpkg.arch.1.gz
-rw-r--r-- 1 root root 32 Dec 9 00:00 dpkg.arch.2.gz
-rw-r--r-- 1 root root 1.5K Dec 8 11:48 dpkg.diversions.0
-rw-r--r-- 1 root root 312 Dec 8 11:48 dpkg.diversions.1.gz
-rw-r--r-- 1 root root 312 Dec 8 11:48 dpkg.diversions.2.gz
-rw-r--r-- 1 root root 213 Oct 21 04:27 dpkg.statoverride.0
-rw-r--r-- 1 root root 162 Oct 21 04:27 dpkg.statoverride.1.gz
-rw-r--r-- 1 root root 162 Oct 21 04:27 dpkg.statoverride.2.gz
-rw-r--r-- 1 root root 767K Dec 11 06:48 dpkg.status.0
-rw-r--r-- 1 root root 195K Dec 10 06:49 dpkg.status.1.gz
-rw-r--r-- 1 root root 195K Dec 8 20:43 dpkg.status.2.gz
drwxr-xr-x 2 root root 4.0K Dec 9 10:55 techstart
```

Figure 12 - Successful execution of the backup script showing creation of a compressed archive.

Automated backups provide a reliable recovery mechanism in the event of accidental deletion, misconfiguration, or security compromise

3.7 Automatic Security Updates

A Bash script was created to perform secure backups of web server files. The script archives critical directories and stores them securely for recovery purposes.

```
azureuser@techstartvm:~$ cat /etc/apt/apt.conf.d/20auto-upgrades
APT::Periodic::Update-Package-Lists "1";
APT::Periodic::Unattended-Upgrade "1";
```

Keeping the system automatically patched reduces the window of exposure to known vulnerabilities and aligns with security best practices.

4. BASH SCRIPTING

To continue on with Bash scripting shown in the previous chapter ill dive into further detail. The custom Bash script was created that automates secure backups of web server data to achieve the goals of security and recovery in the system. If there ever was a system failure the bash script ensures that there is recovery of critical business data in case it is accidentally deleted, as well in the scenario of a system breach. The Bash script I created archives the web app directory and stores it in a secure location on the VM accompanied by a timestamp. I ran the script with after correcting errors and created a zip file archive for a compressed backup. It should be noted that running the script will result in the outputs shown previously in the above paragraph.

The script aids with system resilience in terms of recoverability. Backups act as a reliable solution security wise in the event of a security incident involving security threats like ransomware attacks or simply human error in misconfiguration.

5. ACCESS CONTROL

Access Control was implemented using a defence in depth approach with Azure role based access control, OS permissions, database level access controls and network level restrictions. To ensure that the users only have minimal permissions they need to carry out tasks, the principle of least privilege was used.

RBAC:

RBAC was used on Azure to control access to cloud resources at resource group level. Administrative access was restricted to the project owner, standard users were set to standard roles, ensuring that important resources like virtual machines, virtual networks and databases cant be modified without granted permissions. I didn't create any custom roles, instead stuck with built in Azure roles.

OS Access Controls :

SSH key-based authentication, with password based authentication disabled, was used to restrict access to the virtual machine. Only authorized users were granted permission to access the system and administrative privileges were granted using sudo.

Database Access Control:

SQL authentication with different accounts for administrative and application level access was used to control database access. Administrative actions were restricted to the database administrator account while the application user was granted limited permissions limiting them to only read and write operations that were necessary. This kind of separation prevents the web application from performing privileged database operations.

Blob Storage Access Control:

Azure Blob Storage access was restricted using private networking and access keys. Public access to blob containers was disabled, ensuring that stored data could only be accessed from authorized resources within the virtual network. Encryption at rest was already enabled by default within Azure Blob Storage.

Access Control Matrix:

Team	Azure RBAC Role	OS Access	Database Access	Blob Storage Access
Developers	Contributer	SSH and sudo	Read & Write	Read
QA Testers	Reader	SSH	Read only	Read
Project Managers	Reader	No VM Access	No access	No access

I tested access restrictions across all layers verifying least privilege. Users without authorized roles were forbidden from modifying Azure resources, access the virtual machine and performing authorized database actions. This confirmed access controls were correctly enforced.

6. ADDITIONAL SECURITY MEASURES

Fail2Ban:

Expanding on Fail2Ban, it was deployed on the virtual machine to mitigate SSH brute force attacks. The service continuously monitors authentication logs and automatically bans IP addresses that exceed a set threshold of failed login attempts. This significantly reduces the risk of automated attacks. Fail2Ban operates independently of network-level controls, providing host-based intrusion prevention and reinforcing the system's defense in depth strategy.

Private Endpoints:

Private endpoints were set up to limit access to Azure SQL Database and Azure Blob Storage to the internal virtual network. By removing them from public access, it reduces the risk associated with lateral movement and also reduces the attack surface. Private end points will be used to set up Azure SQL Database and Azure Blob Storage. It will reduce attacks associated with Azure Blob Storage.

7. CHALLENGES

Private Enpoints:

During the deployment and security setup phase, there were some technical problems that were tackled. Handling these problems not only aided in understanding cloud networking and security but also cloud system integration. As a result of enabling private endpoints on Azure SQL Database, direct connections with external tools were not working. At first, it made it hard for a successful database connection from the virtual machine. The problem was fixed by ensuring that the VM was properly put into the same virtual network and subnet and then verifying that the DNS resolution for private endpoints was properly configured. Once done, it was possible to access the database as needed while being isolated from the public network.

Web Application:

During the deployment of the Flask web app, it was initially difficult to access the login webpage via the browser, even with all services running properly on the virtual machine. It was discovered that there were limitations imposed by firewalls on either side at UFW or NSG. By opening only the necessary port for the required application and checking listening services using system tools, it ensured that external

accessibility was reached without opening any security gaps. It once again showed the need for proper firewall management.

HTTPS:

Use of HTTPS with a certificate signed within the organization led to security warnings on browsers, and at first, there was a chance of a misconfiguration. I then found out that the warning messages were expected because there was no certificate authority. It made it clear that there should be a difference between an encrypted communications relationship and trust for a certificate.

These problems have shown that planning, testing, and troubleshooting were very essential. By overcoming these problems, it helped tighten up the security configuration and build confidence in cloud infrastructure management.

10. CONCLUSION

In conclusion, a successful implementation of a cloud based web application hosting on Microsoft Azure and incorporating a broad set of security controls on various levels, including network, operating system, database, and application levels, was carried out. A defense in depth strategy was used at all levels so that no single security solution will be dependent on another.

The main security needs were addressed using Network Security Groups and host based firewalls for network traffic control, SSH Key Authentication and PAM capabilities for securing system access, and HTTPS/TLS and Azure SQL Transparent Data Encryption capabilities for encrypting data. Automated updates, logging, and monitoring capabilities added extra security layers to the system by addressing vulnerabilities and providing insights into security-related activities.

Access control was maintained with Azure Role-Based Access Control, operating system permissions, and role assignments at the database level. Overall, there was strict enforcement of the principle of least privilege. Different accounts and roles were maintained for administrative and app access. Also, private endpoints were employed to decrease the attack surface associated with accessing private services.

Aside from these required security measures, additional steps were also considered and made, such as using Fail2Ban and automated scripts for backups. These actions helped me in understanding and threats and operational security aside from getting it done and deployed. This project has been very useful in offering a hands on learning approach on cloud infrastructure security and learning about the

various aspects of cloud security best practices. The steps implemented addresses all project requirements and cloud security best practices.